

Recall: RFLAGS register

(it will play a role in control flow)

- RFLAGS contain different status flags
 - ZF, SF, CF, OF
- Certain instructions set status flags
 - Regular arithmetic instructions
 - Special flag-setting instructions
- Instructions that read RFLAGS to...
 - set register values
 - determine value of %rip

Today's lesson plan

- Special instructions that set RFLAGS
 - `cmp`, `test`
- Instructions that read RFLAGS to set register values
 - `set`
- Instructions that (read RFLAGS to) set `%rip`
 - `jmp`

Status flags summary

flag	status
ZF (Zero Flag)	set if the result is zero.
SF (Sign Flag)	set if the result is negative.
CF (Carry Flag)	Overflow for unsigned-integer arithmetic
OF (Overflow Flag)	Overflow for signed-integer arithmetic

Arithmetic instructions set RFLAGS, e.g. add, inc, and, sal

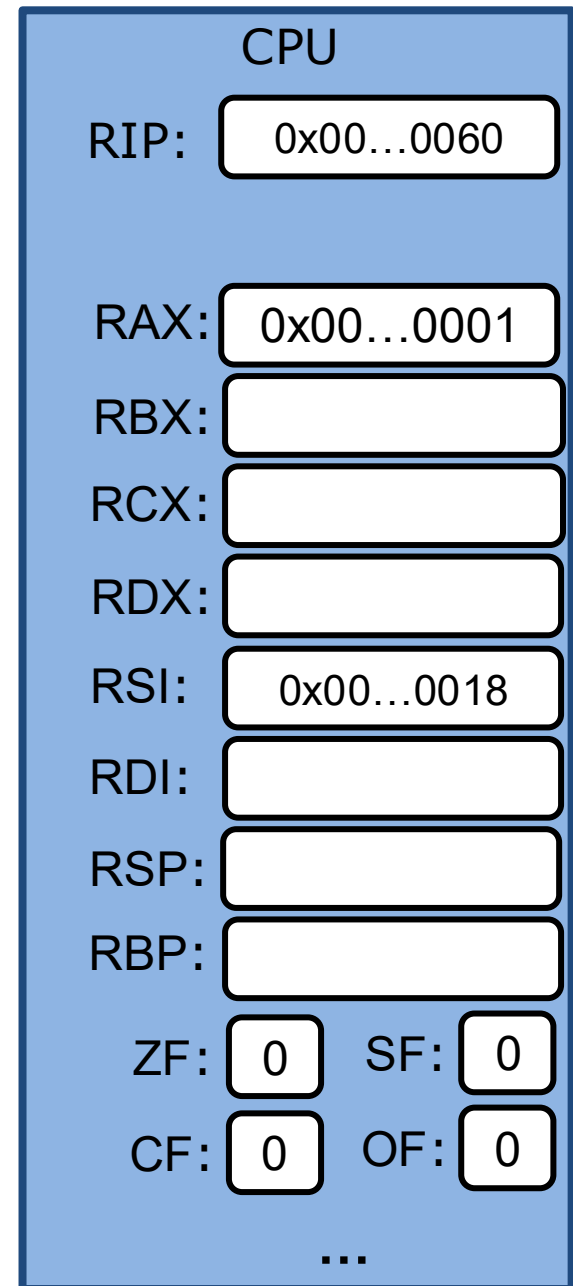
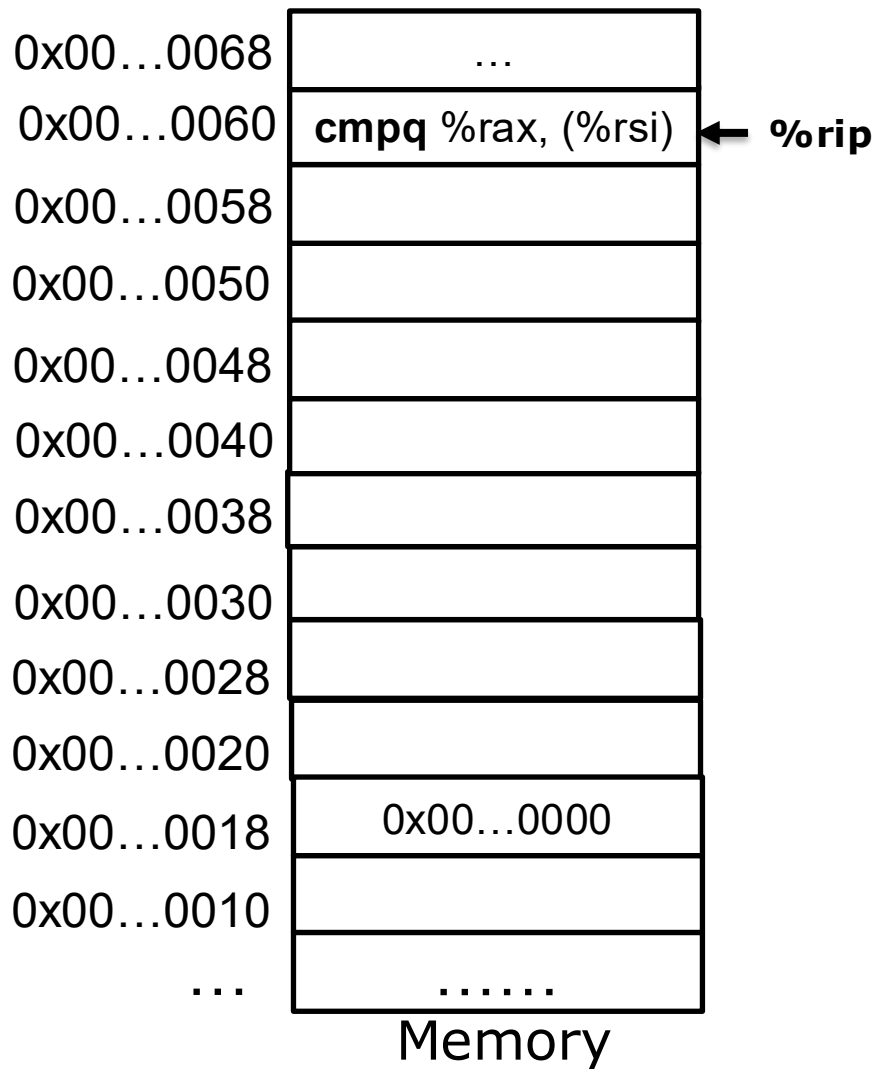
lea, mov do not set RFLAGS

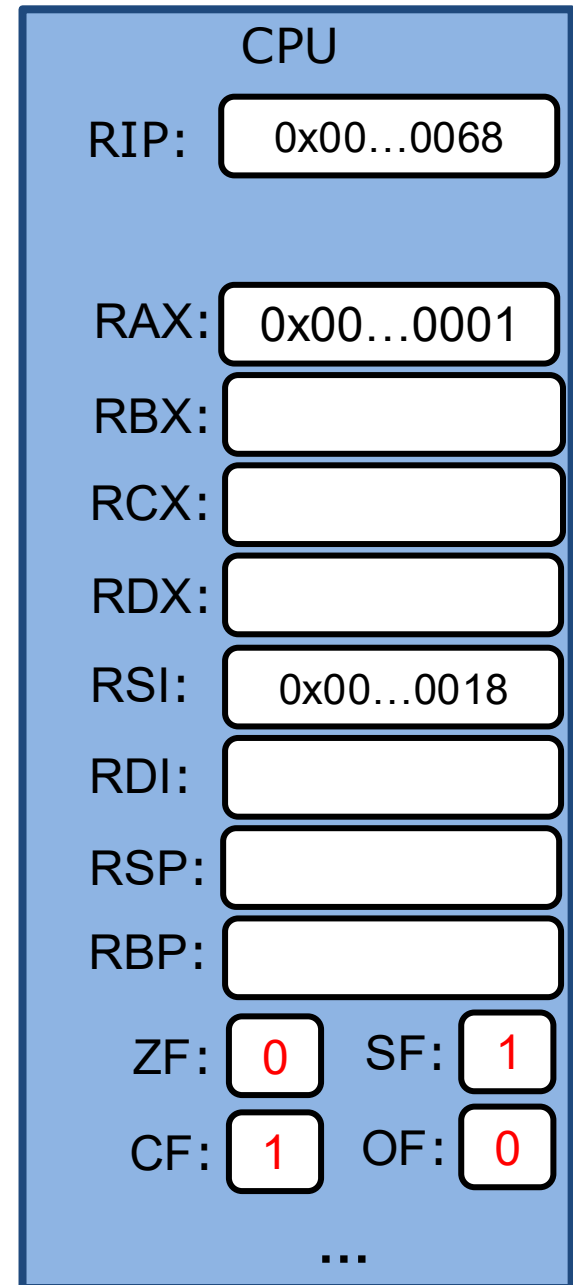
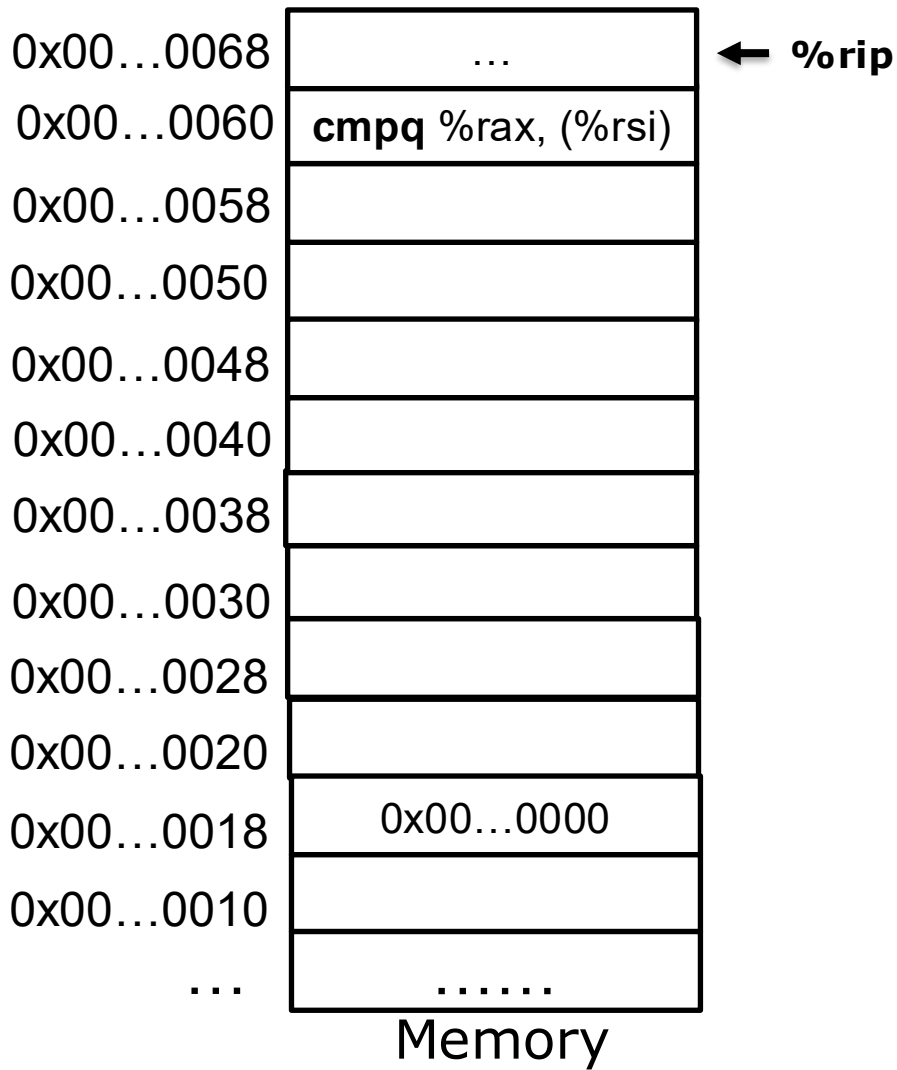
Special instructions that set RFLAGS:

cmp

cmpq src, dst

- Set CF, ZF, SF and OF like **subq** src, dst except dst is unchanged





Special instructions that set RFLAGS: **test**

testq src, dst

- Set ZF, SF like **andq** src, dst except dst is unchanged

Questions

testq %rax, %rax

- When is ZF set?
- When is SF set?

Questions

testq %rax, %rax

- When is ZF set? $\text{val}(\%rax) = 0x0$
- When is SF set? $\text{val}(\%rax) < 0$

Instructions that read RFLAGS: set

set**X** dst

- Set dst to 1 (or 0) if condition is true (or false).
- Suffix (**X**) indicates which condition to test for
 - Truthfulness of condition depends on status flags in RFLAGS.
- dst is a 1-byte register or a byte in memory.

setX dst

```
cmpq a, b  
setX dst
```

SF:true, OF:false

SF:false, OF:true

← cmpq a=0x80...0, b=0xff.f

setX	Condition	Description
sete	ZF	Equal / Zero
setne	~ZF	Not Equal / Not Zero
sets	SF	Negative
setns	~SF	Nonnegative
setg	~ (SF^OF) & ~ZF	Greater (Signed)
setge	~ (SF^OF)	Greater or Equal (Signed)
setl	(SF^OF)	Less (Signed)
setle	(SF^OF) ZF	Less or Equal (Signed)
seta	~CF & ~ZF	Above (unsigned)
setb	CF	Below (unsigned)

b < a

Example

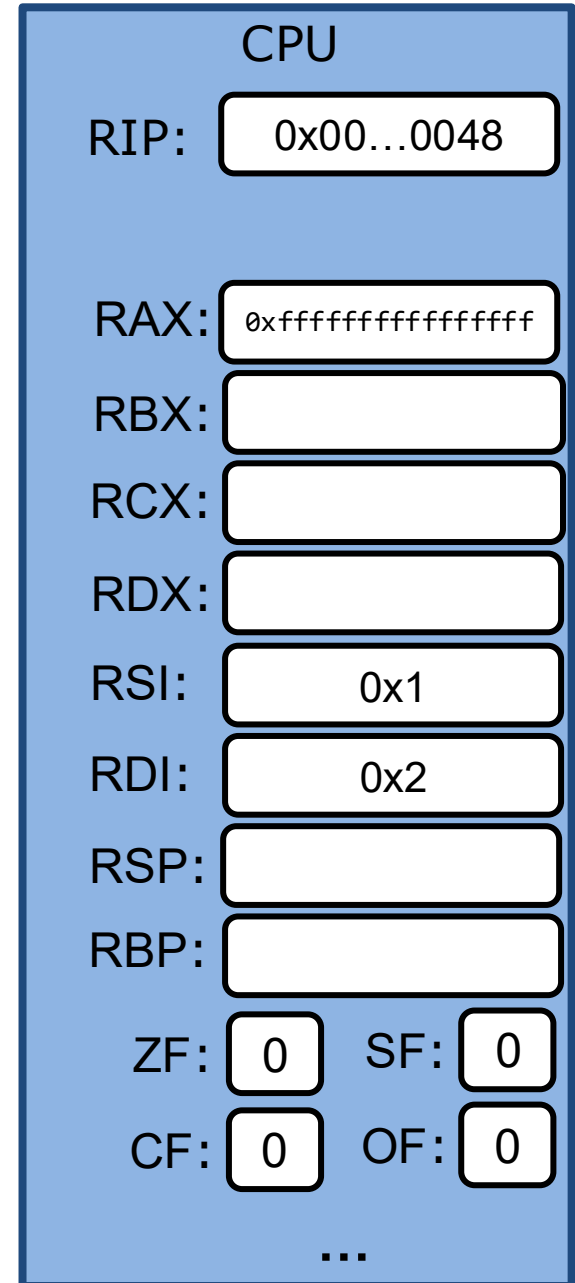
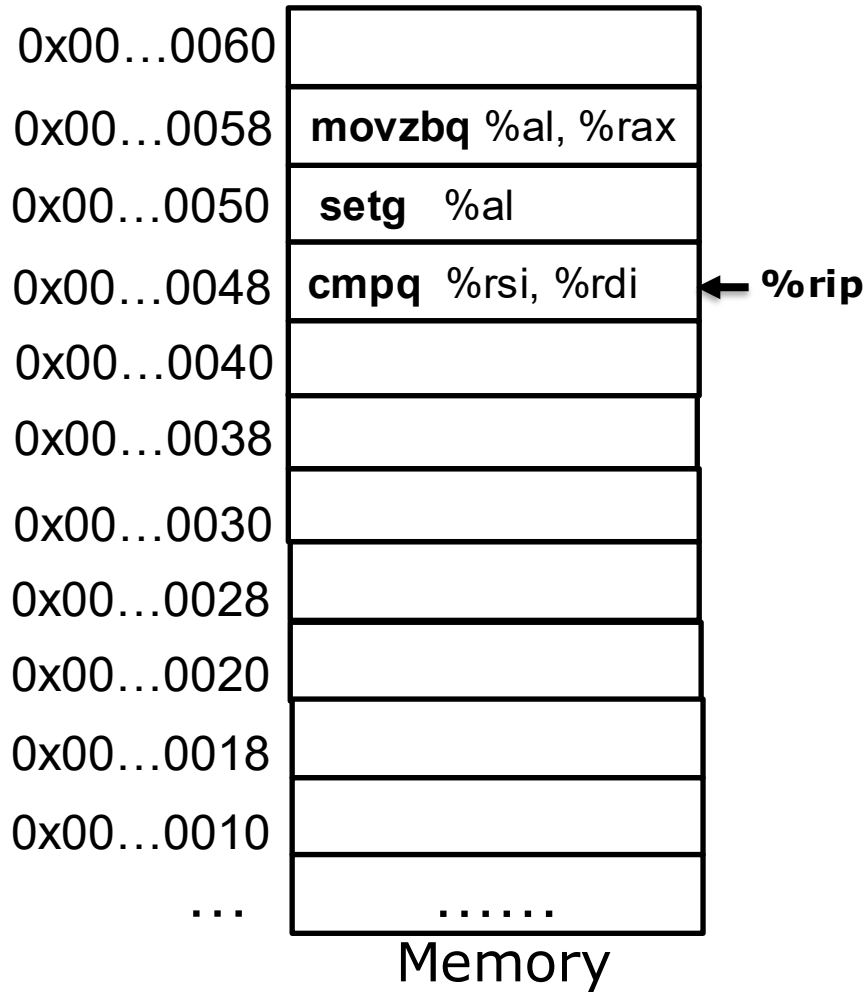
```
long gt (long x, long y)
{
    return x > y;
}
```

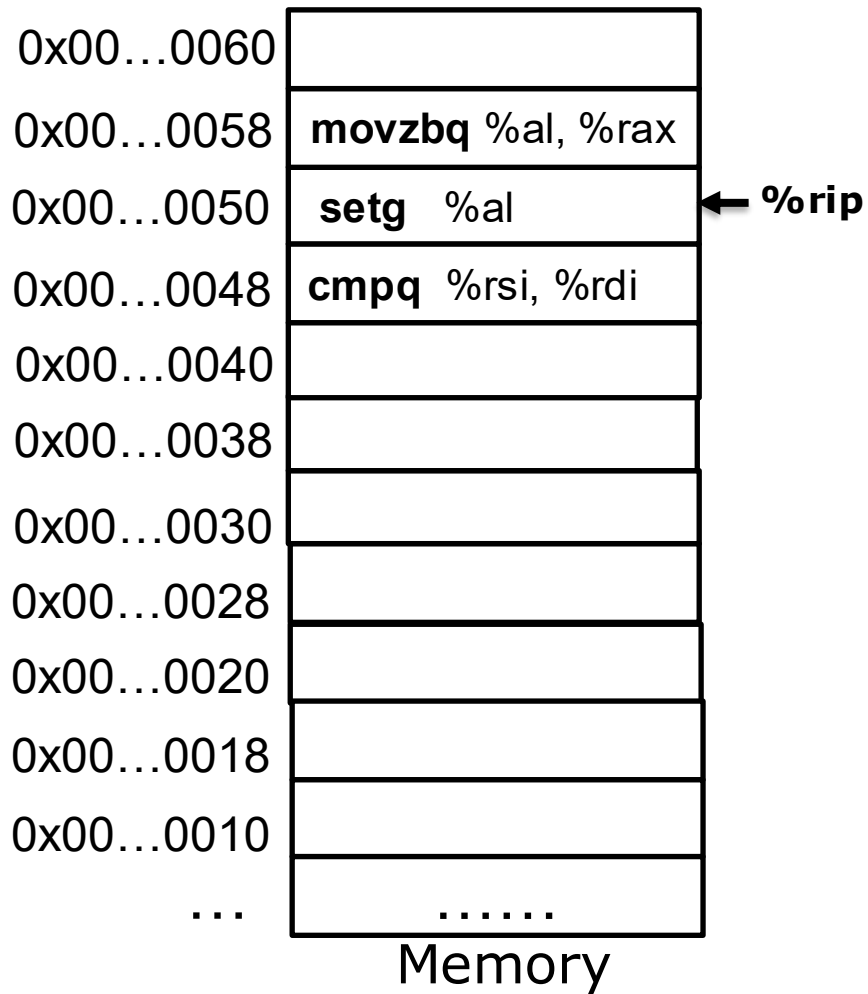
Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value



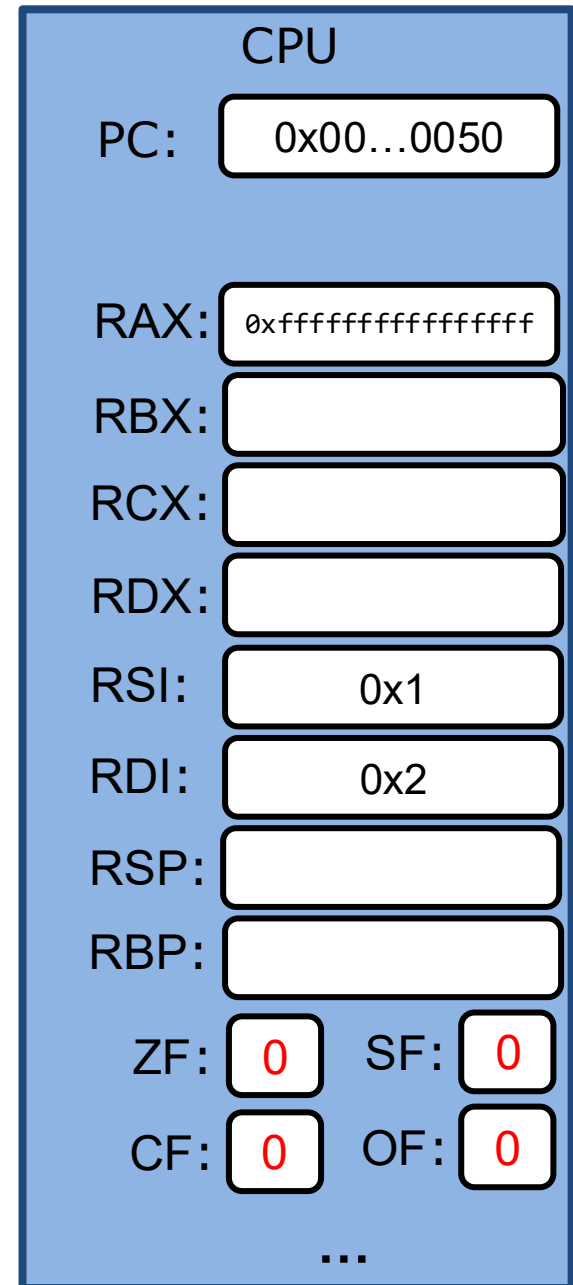
Least significant byte of %rax register

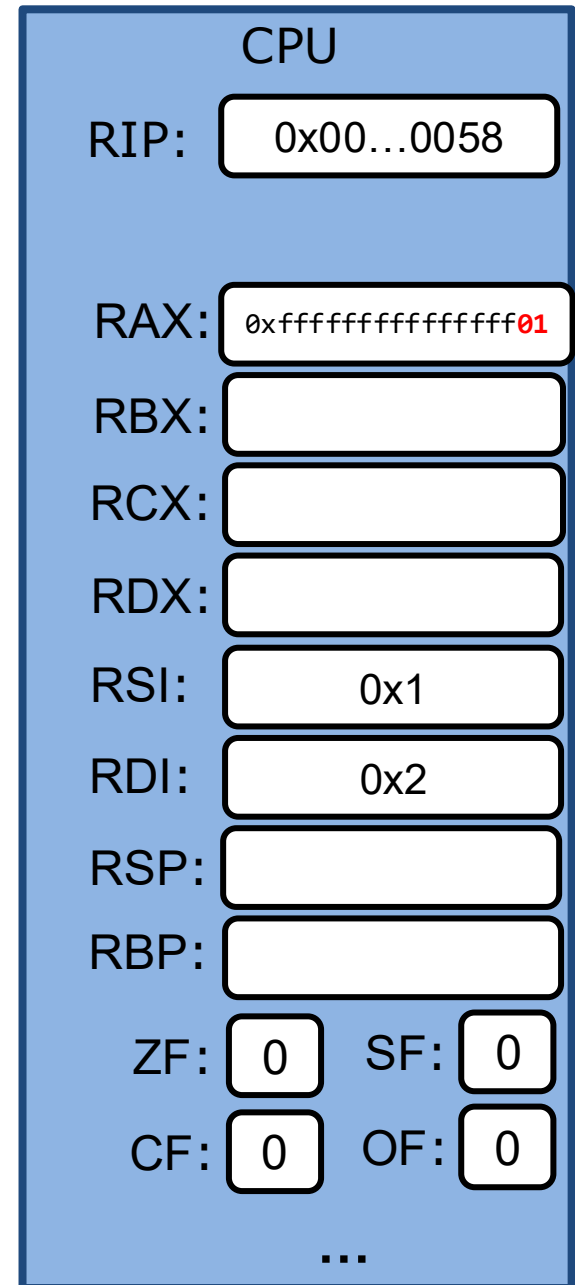
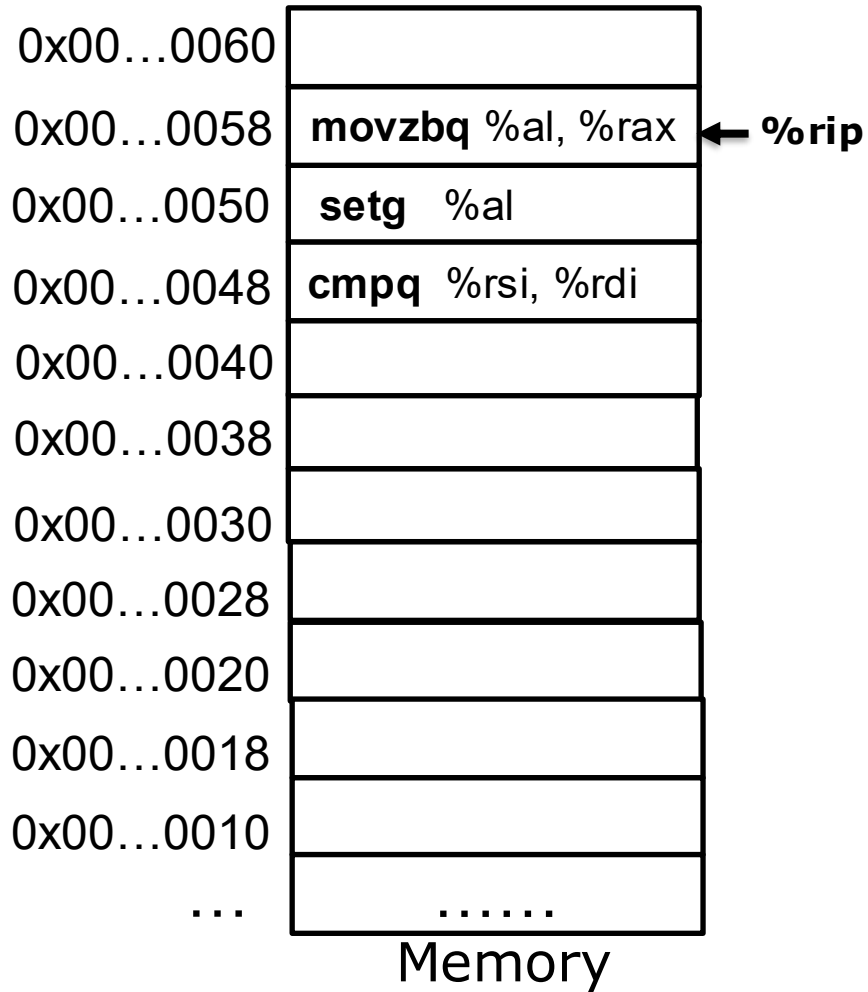
```
cmpq    %rsi, %rdi    # cmpq y x
setg    %al          # set when >
movzbq  %al, %rax      # zero extend %rax
```

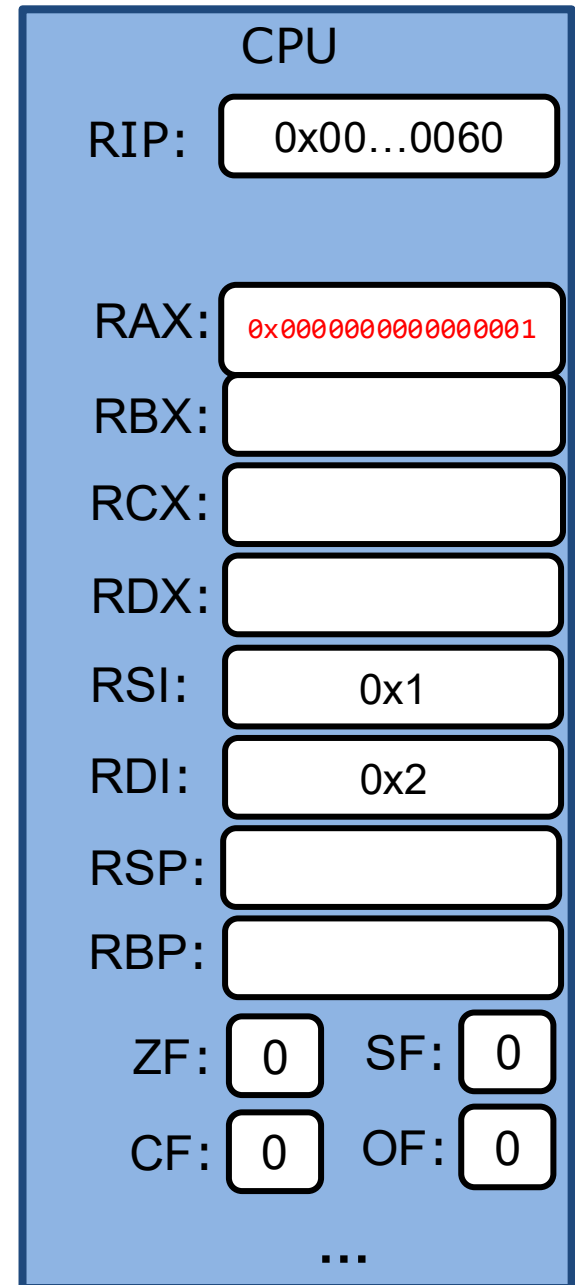
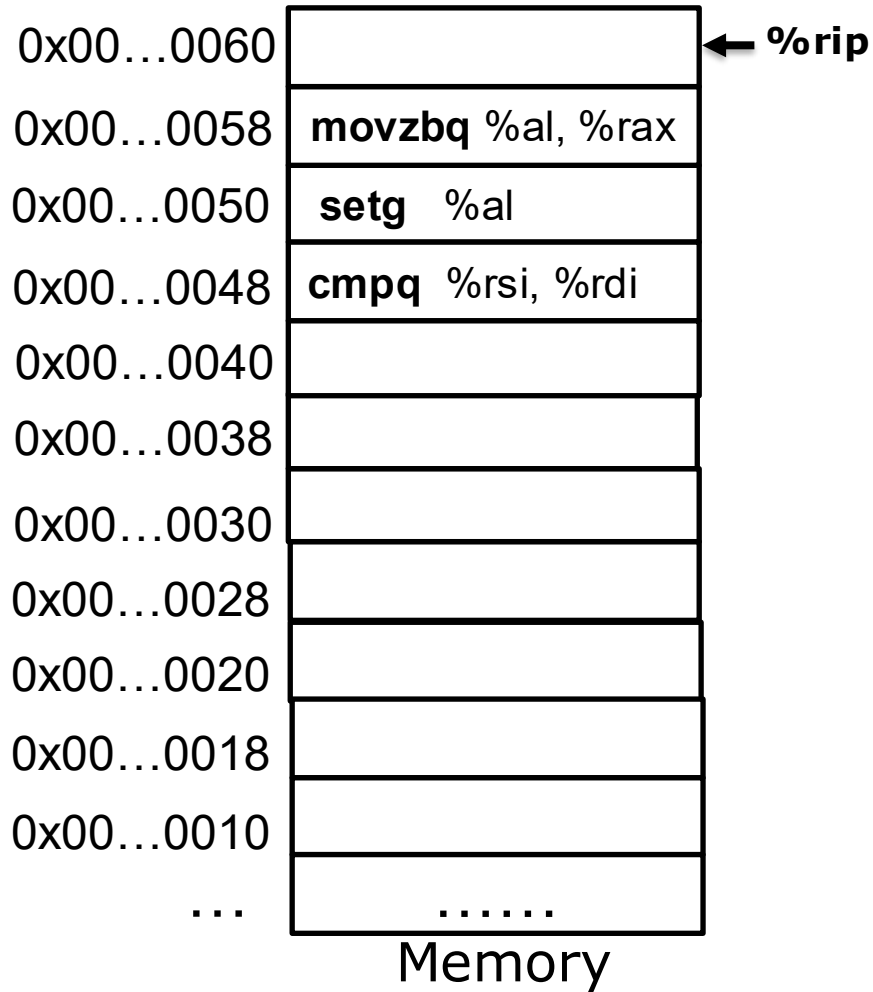




setg	~ (SF^OF) & ~ZF
------	-----------------







Today's lesson plan

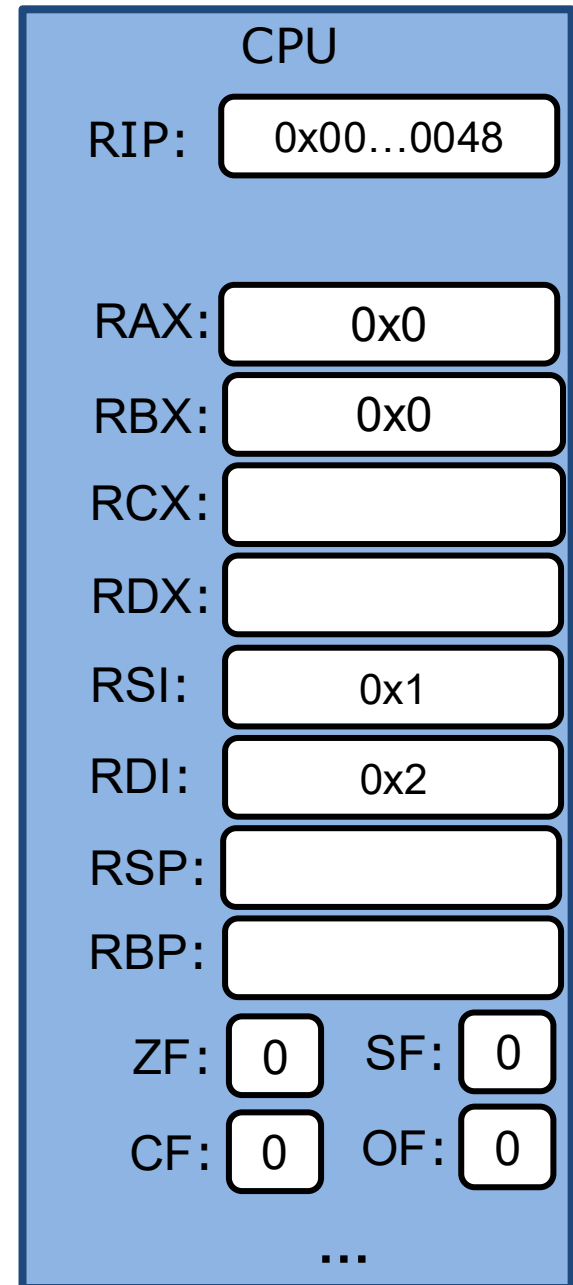
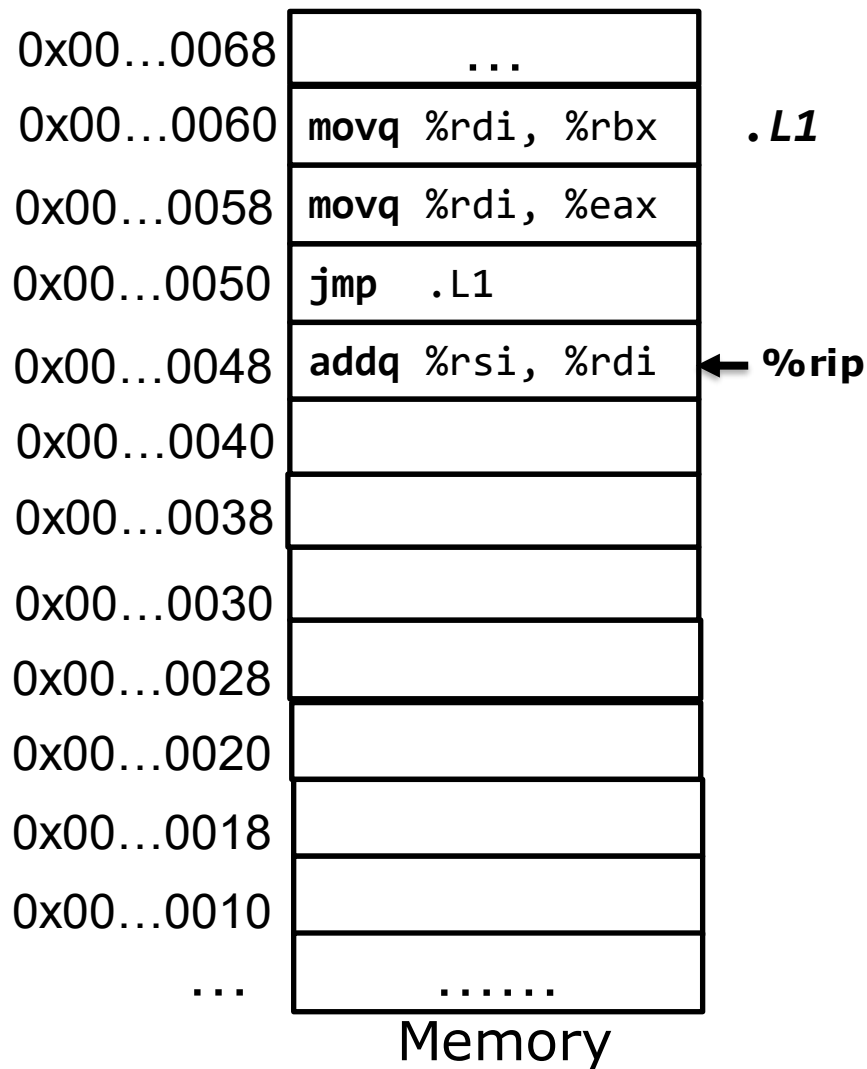
- Special instructions that set RFLAGS
 - Cmp, test
- Instructions that read RFLAGS to set register values
 - Set
- Instructions that (read RFLAGS to) set %rip
 - jmp

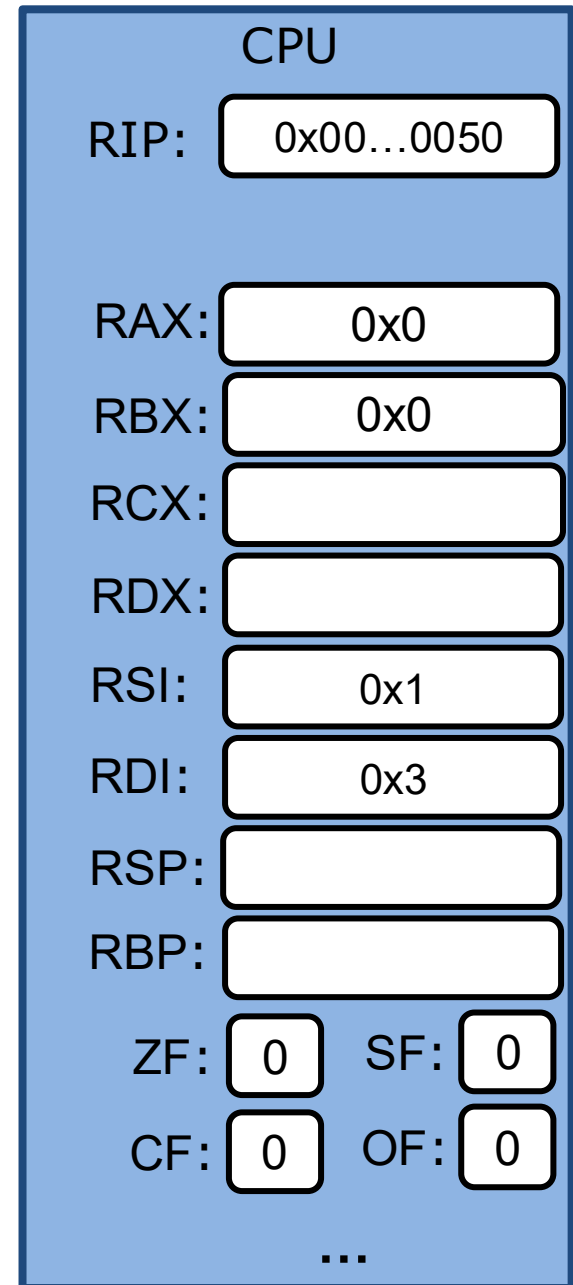
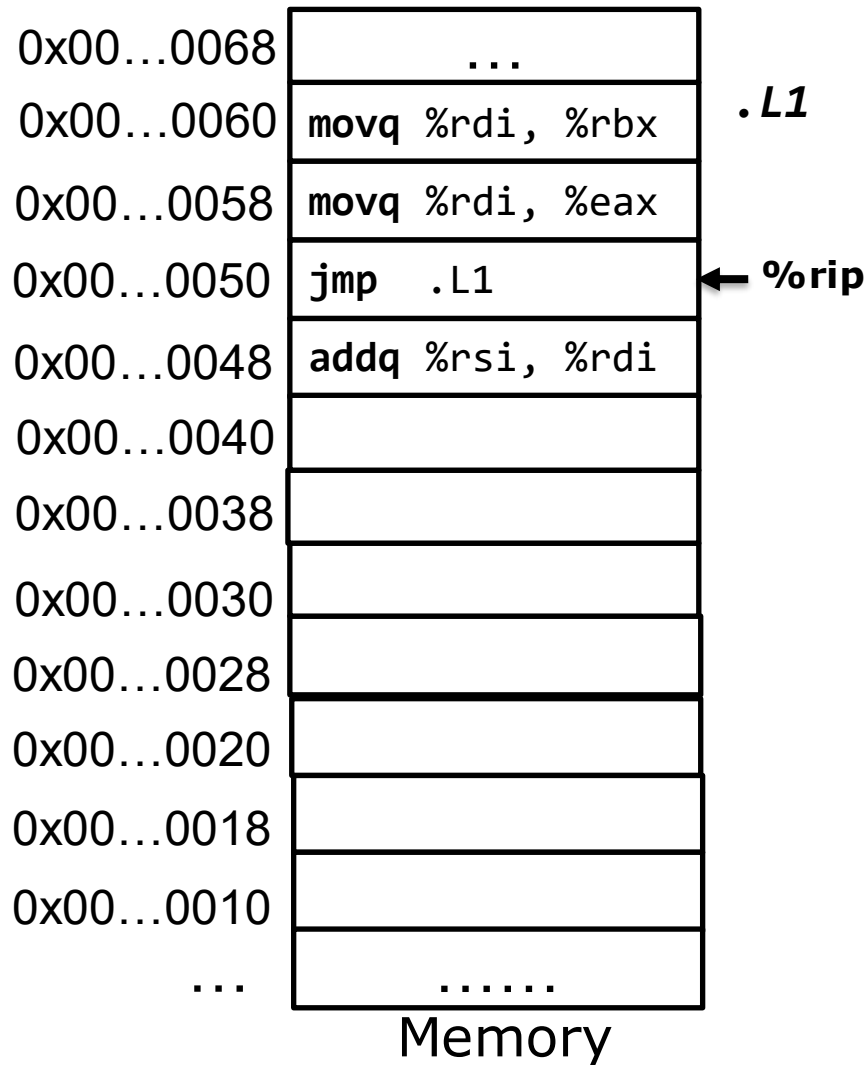
(Unconditional) jump instruction

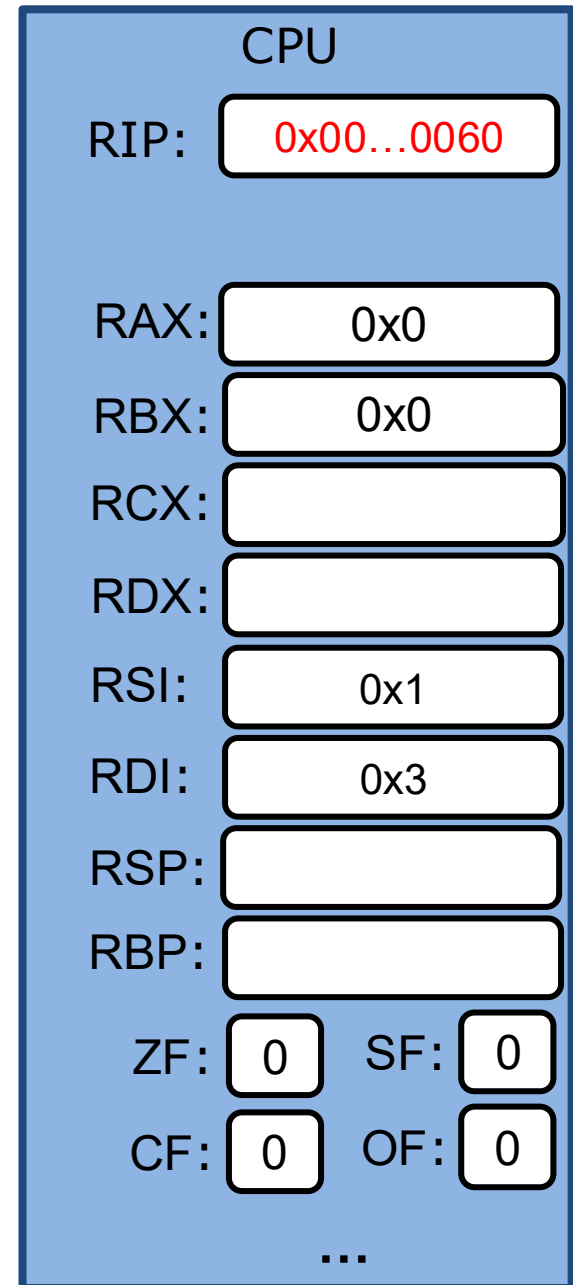
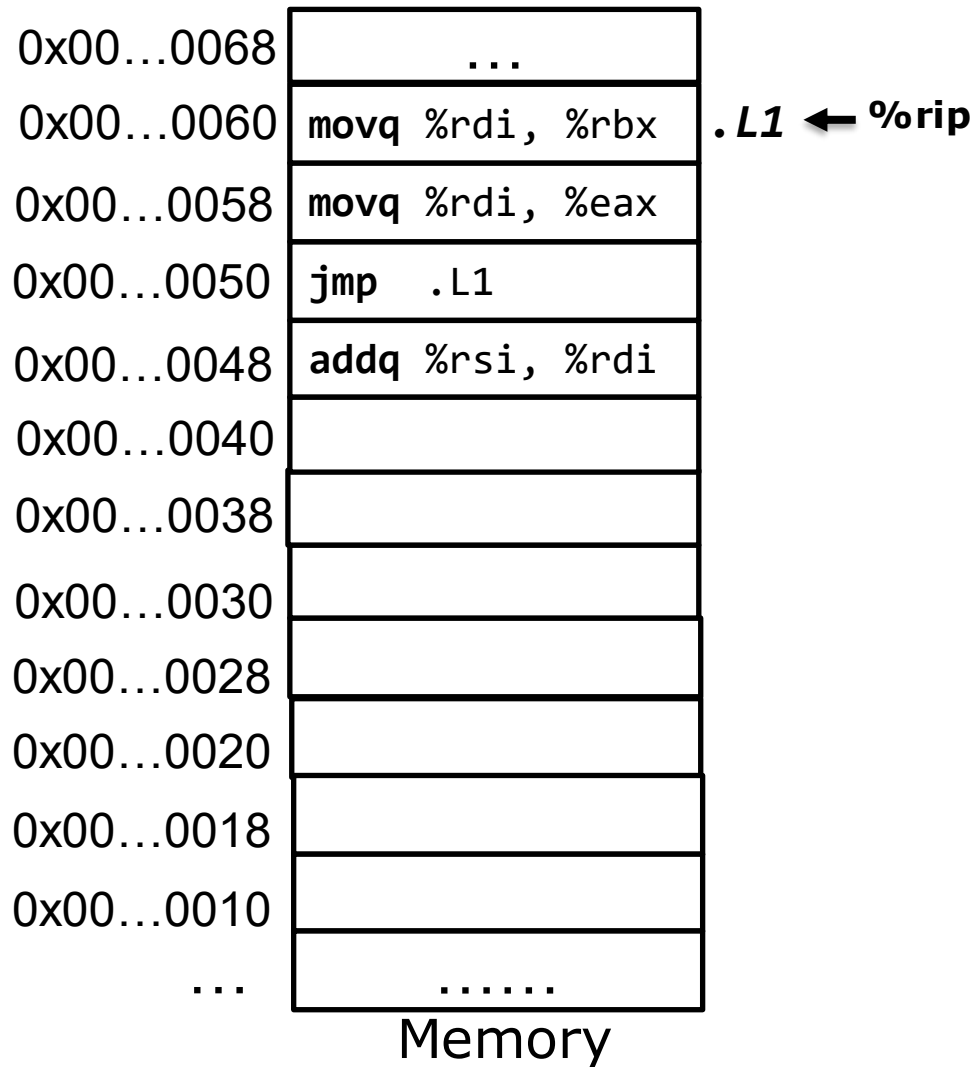
jmp label

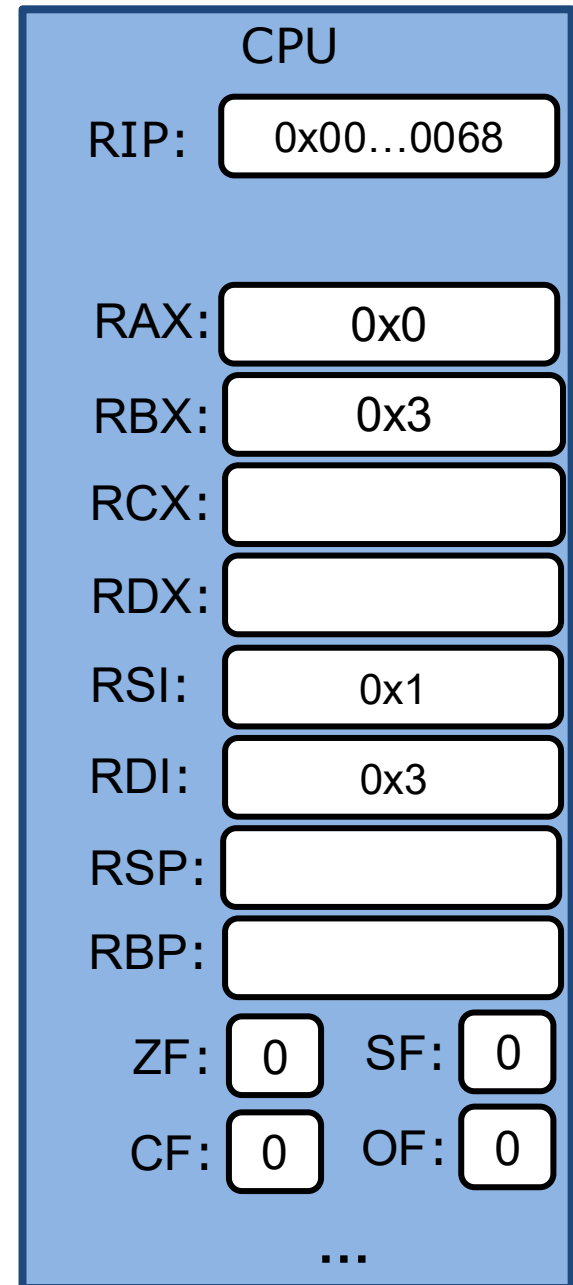
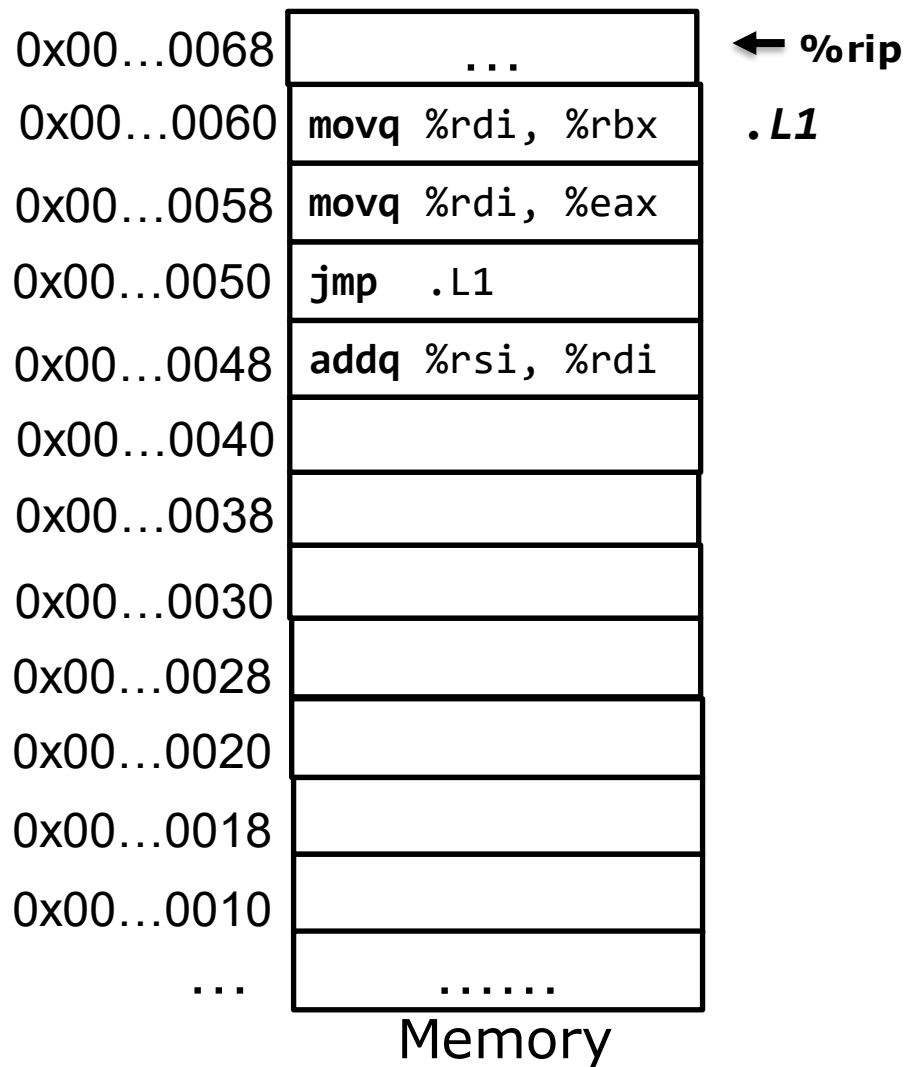
- Change %rip to the address specified by label
- jmp is like *goto*

```
addq %rsi, %rdi
jmp  .L1
movq %rdi, %eax
.L1
movq %rdi, %rbx
```







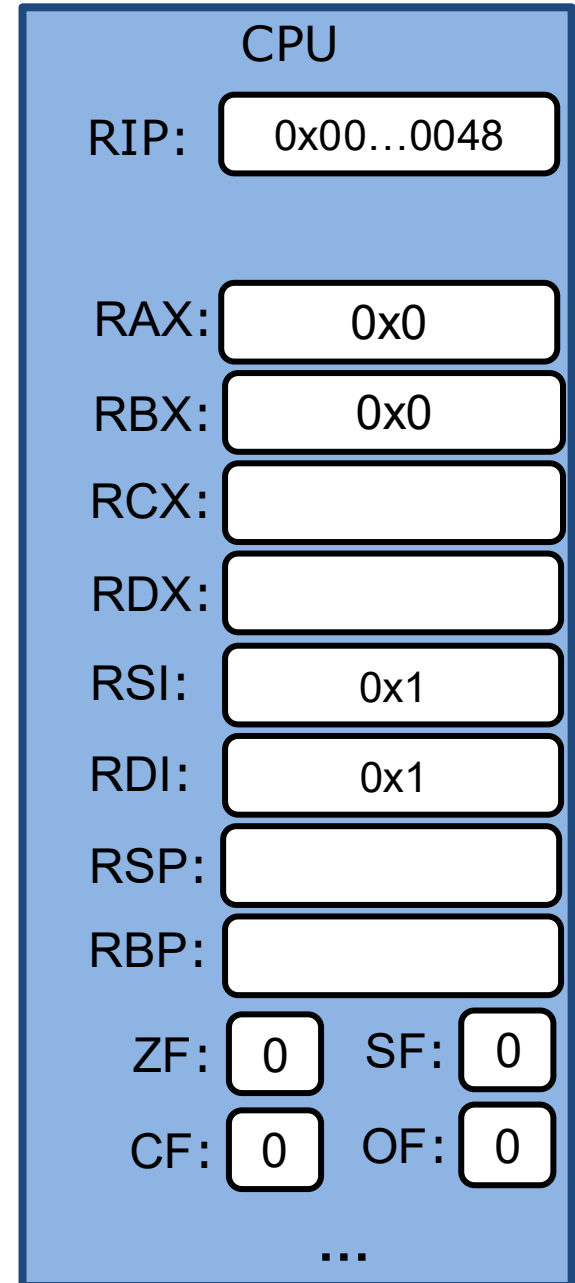
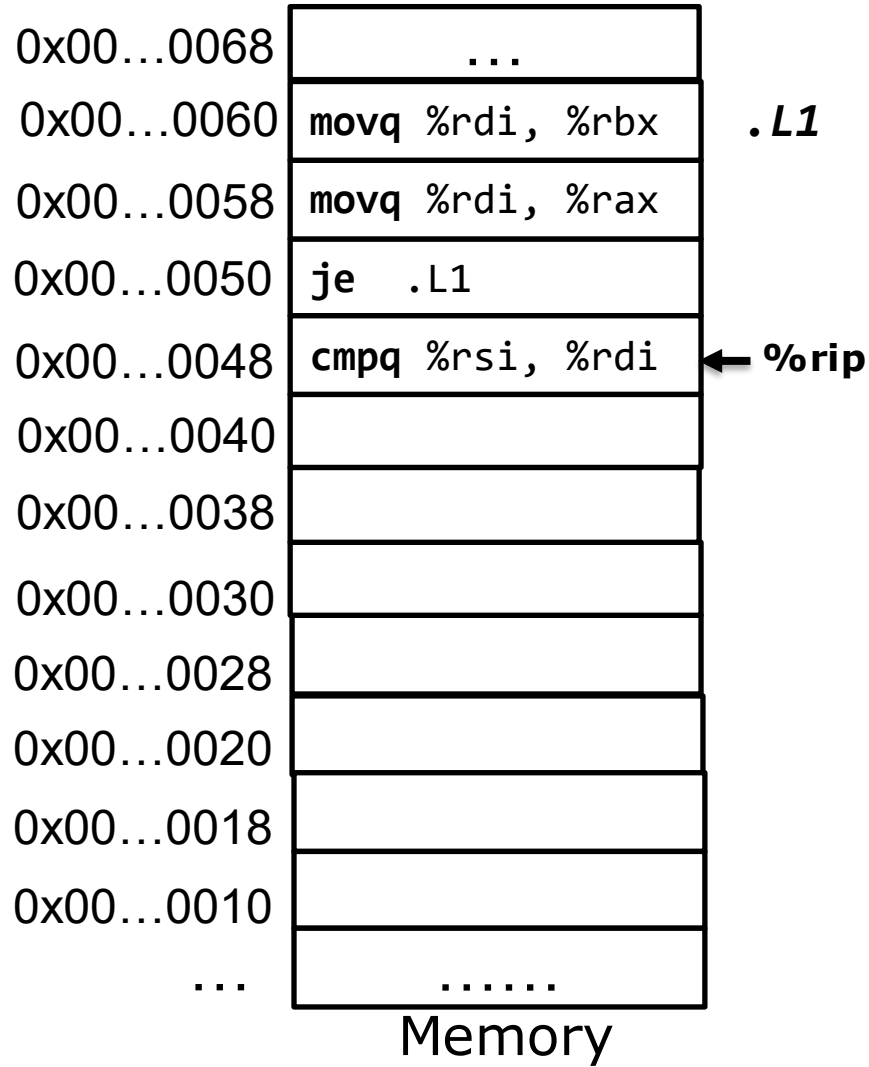


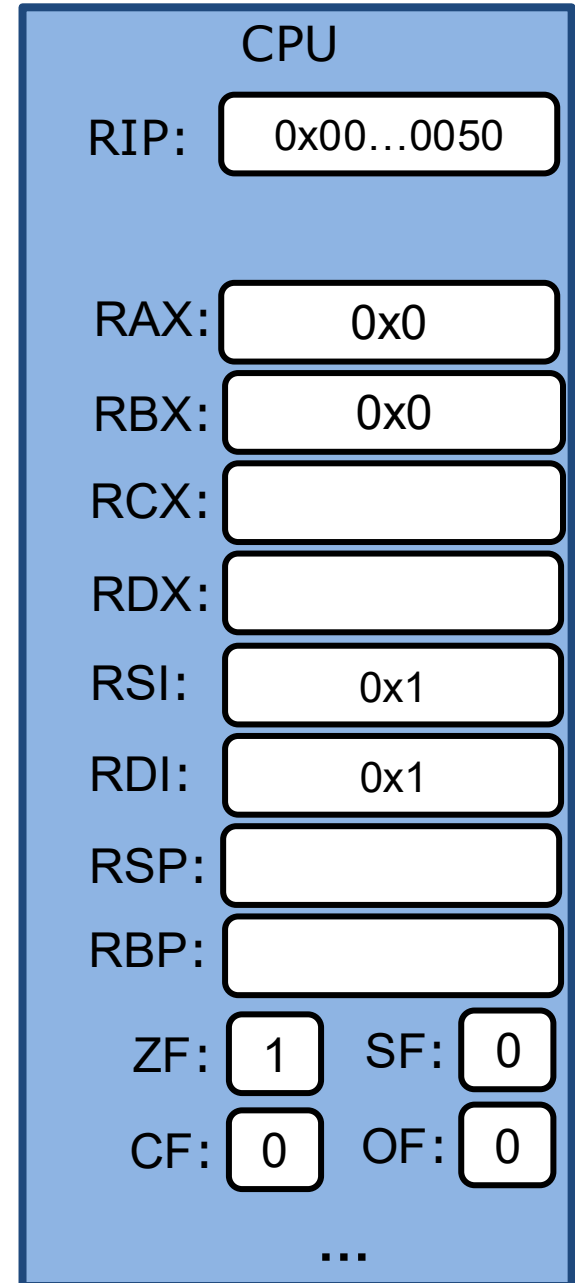
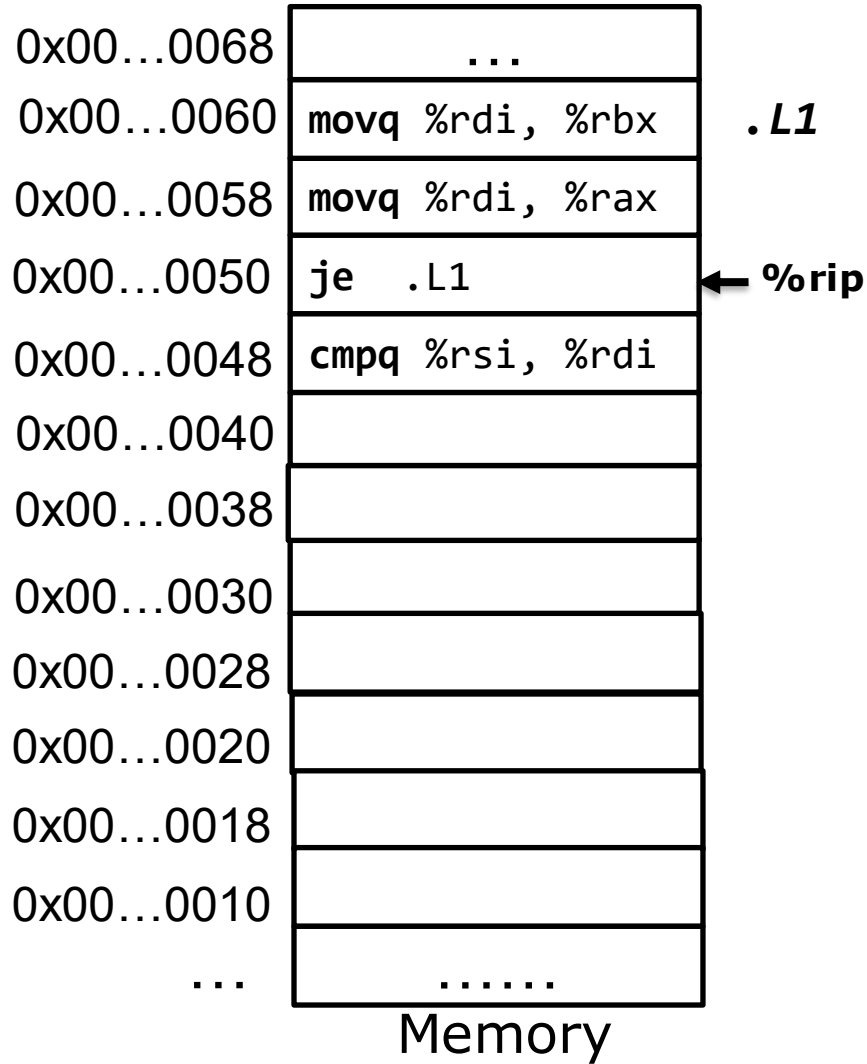
Conditional jump instruction

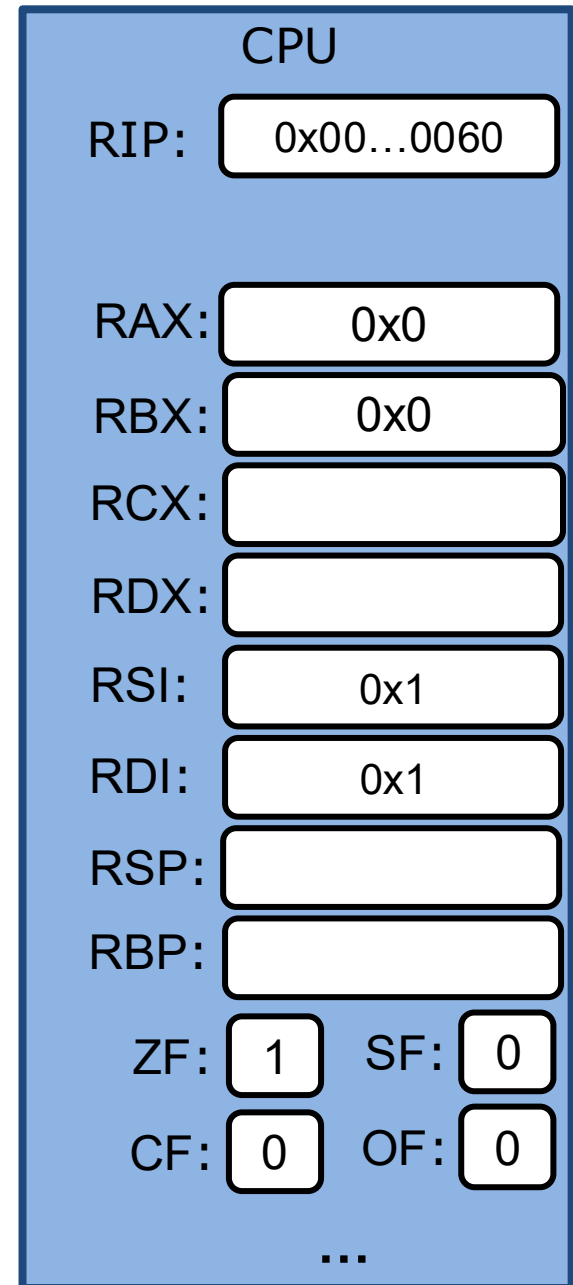
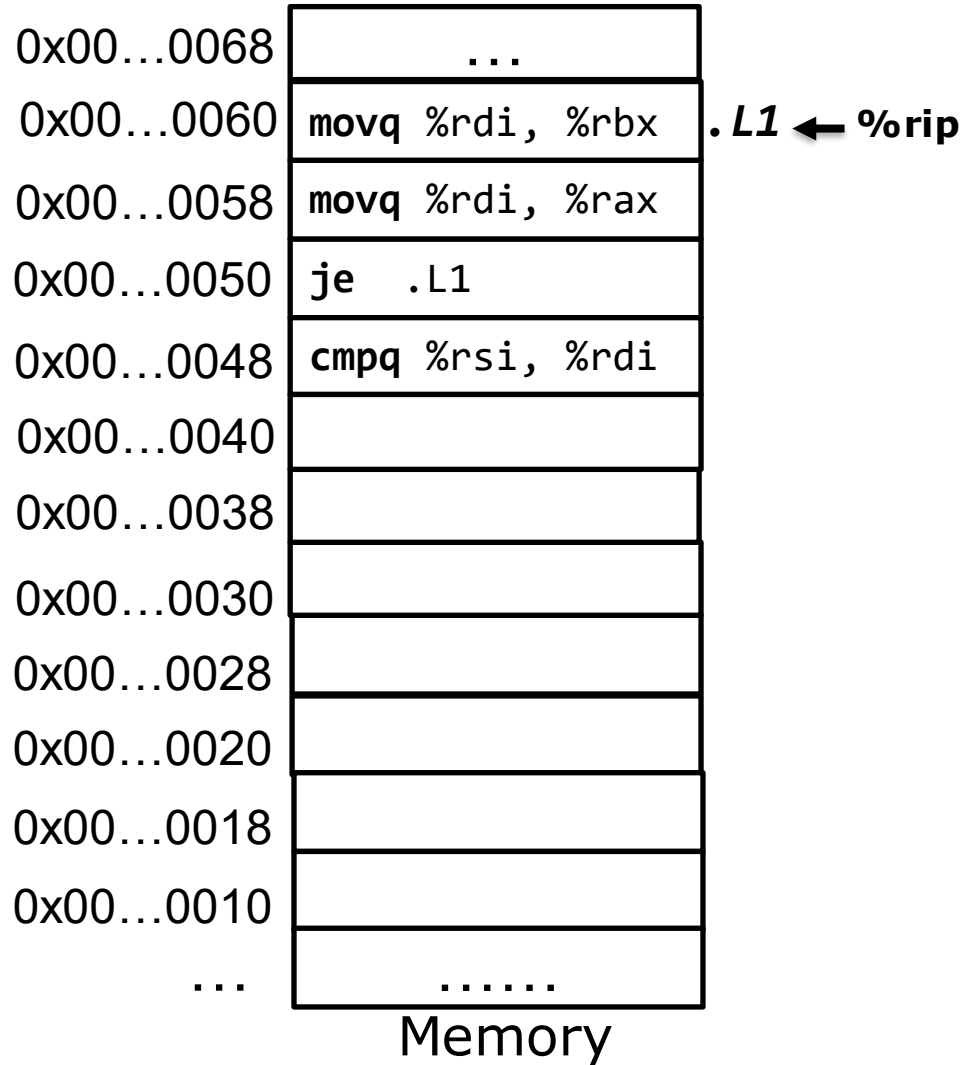
jX label

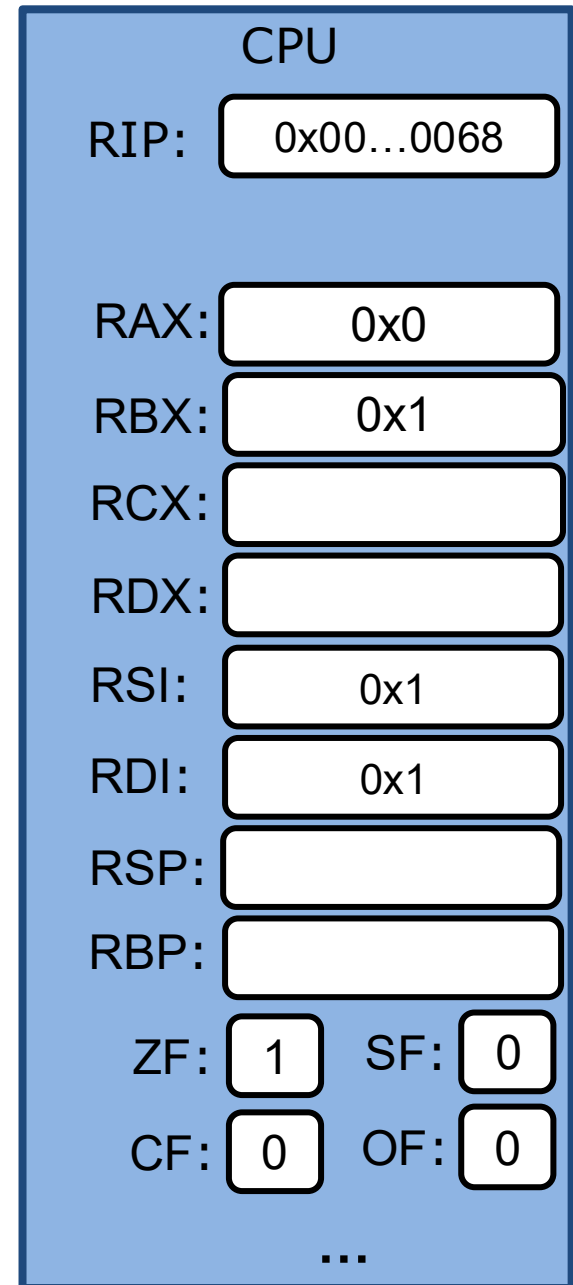
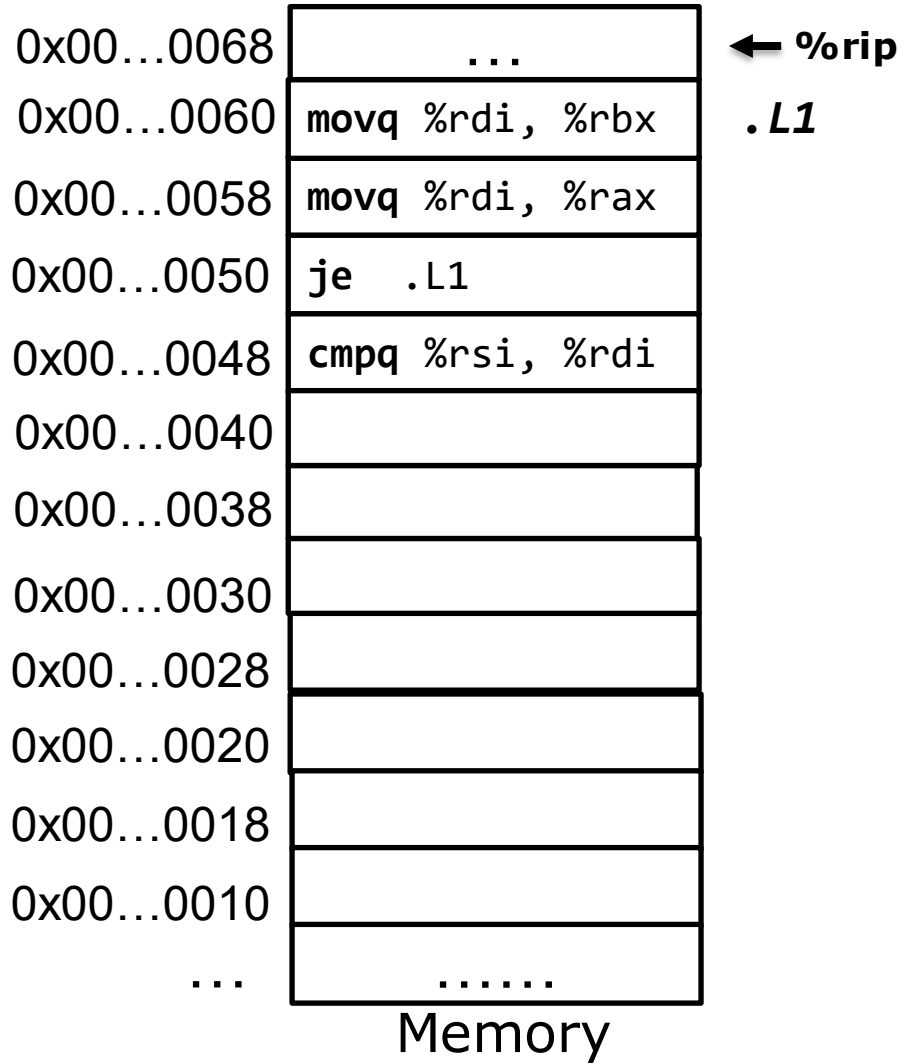
- If condition **X** is met, jump to the label

jX	Condition	Description
j_e	ZF	Equal / Zero
j_{ne}	~ZF	Not Equal / Not Zero
j_s	SF	Negative
j_{ns}	~SF	Nonnegative
j_g	~ (SF^OF) & ~ZF	Greater (Signed)
j_{ge}	~ (SF^OF)	Greater or Equal (Signed)
j_l	(SF^OF)	Less (Signed)
j_{le}	(SF^OF) ZF	Less or Equal (Signed)
j_a	~CF & ~ZF	Above (unsigned)
j_b	CF	Below (unsigned)









How “if..else..” statement works

```
long compare(long x, long y)
{
    long result;
    if (x > y)
        result = 1;
    else
        result = 0;
    return result;
}
```

gcc -Og -S compared.c

```
compare:
    cmpq    %rdi, %rsi
    jge     .L3
    movl    $1, %rax
    ret
.L3:
    movl    $0, %rax
    ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rax	Return value

How “while” works

```
long count(unsigned long x)
{
    long cnt = 0;
    while (x != 0) {
        x = x >> 1;
        cnt++;
    }
    return cnt;
}
```

gcc -Og -S count.c

logical right shift

```
count:
    movq $0, %rax
    jmp .L2
.L3:
    shrq %rdi
    addq $1, %rax
.L2:
    testq %rdi, %rdi
    jne .L3
    ret
```

Sets RFLAGS based on
result of logical AND

Register	Use(s)
%rdi	Argument x
%rax	Return value

“While” Translation example

```
long count(unsigned long x)
{
    long cnt = 0;
    while (x != 0) {
        x = x >> 1;
        cnt++;
    }
    return cnt;
}
```

count:

```
movq $0, %rax
```

```
jmp .L2
```

.L3:

```
shrq %rdi
```

```
addq $1, %rax
```

.L2:

```
testq %rdi, %rdi
```

```
jne .L3
```

```
ret
```

```
long cnt = 0;
goto .L2
```

.L3:

```
x = x >> 1
cnt = cnt + 1
```

.L2:

```
if x != 0
    goto .L3
```

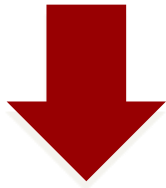
```
return cnt
```

Register	Use(s)
%rdi	Argument x
%rax	Return value

"For" Loop translation

For Version

```
for (Init; Test; Update )  
    Body
```



While Version

```
Init ;  
while (Test) {  
    Body  
    Update ;  
}
```

“Loop” Translation example

- **gcc -Og -S *.c**

```
long sum(long n)
{
    long s = 0;
    for (long i=0; i<n; i++){
        s += i;
    }
    return s;
}
```

```
sum:
    movq $0, %rdx
    movq $0, %rax
    jmp .L5
.L6:
    addq %rdx, %rax
    addq $1, %rdx
.L5:
    cmpq %rdi, %rdx
    jl  .L6
    ret
```

Register	Use(s)
%rdi	n
%rax	Return value (s)

Summary

- How CPU implements non-linear control flow
- Special register RFLAGS encodes status flags (ZF, SF, CF, OF)
- `cmp` (or `test`) followed by `jmp` (or `set`)